

Lauffähiger Code

<https://github.com/yaniktf/iv-repo>

Ausführung

```
cd 9
elm reactor
```

Anschließend im Browser `http://localhost:8000` öffnen und dort `Main.elm` auswählen.

Datenstruktur und Layout

Jeder Knoten wird einheitlich als `Node` modelliert; eine eindeutige ID wird nach dem Aufbau über `Tree.indexedMap` vergeben (für das Hervorheben beim Hovern):

```
type alias Node =
  { id : Int, label : String, value : Maybe Int }

assignIds : Tree Node -> Tree Node
assignIds =
  Tree.indexedMap (\id node -> { node | id = id })
```

Das Layout berechnet `Hierarchy.tidy`. Es liefert für jeden Knoten die Koordinaten `x/y` sowie `width/height` zurück. Die eigentliche Zeichenaufgabe – Knoten als `circle`, Kanten als `line` – wird selbst implementiert:

```
positionedTree = Hierarchy.tidy layoutAttrs sourceTree
positionedNodes = Tree.toList positionedTree
links          = Tree.links positionedTree

linkElement ( parent, child ) =
  line [ x1 parent.x, y1 parent.y, x2 child.x, y2 child.y ] []
```

Aufgabe 9.1: Binärer und geordneter Baum

Beide Bäume werden mit `Tree.tree` bzw. dem Helfer `leaf` (für Blätter) direkt im Code aufgebaut. Der binäre Baum bildet Folie 405 nach, der geordnete Baum Folie 412:

```
orderedTree =
  assignIds
    (Tree.tree (mkNode "root" Nothing))
```

```
[ Tree.tree (mkNode "group-a" Nothing)
  [ leaf "a1", leaf "a2", ... ]
, leaf "single-b"
, ...
] )
```

Für diese kleinen Bäume wird das Layout über `Hierarchy.size 1320 520` fest auf die Zeichenfläche skaliert. Knoten erhalten den Radius 20, Labels werden hier nicht angezeigt (nur beim Hovern).

Aufgabe 9.2: flare.json laden und darstellen

Die Datei wird beim Start per `Http.get` geladen und über `Cmd.batch` parallel zur Länderdatei angefordert. Ein rekursiver Decoder überführt das geschachtelte JSON in einen `Tree Node`. Ein Knoten hat entweder `children` (innerer Knoten) oder einen `value` (Blatt):

```
flareDecoder : Decoder (Tree Node)
flareDecoder =
  Decode.map3
    (\name value children ->
      Tree.tree (mkNode name value) (Maybe.withDefault [] children))
    (Decode.field "name" Decode.string)
    (Decode.maybe (Decode.field "value" Decode.int))
    (Decode.maybe
      (Decode.field "children"
        (Decode.list (Decode.lazy (\_ -> flareDecoder)))))
```

`Decode.lazy` ist nötig, da der Decoder sich für die Kinder selbst aufruft. `Decode.maybe` fängt fehlende Felder ab, sodass derselbe Decoder für innere Knoten und Blätter funktioniert.

Labels um 90 Grad drehen

Da der Baum sehr breit ist, würden waagerechte Labels überlappen. Sie werden deshalb wie gefordert um 90° um ihren eigenen Ankerpunkt gedreht, sodass sie senkrecht und gut lesbar stehen:

```
rotation =
  if config.rotateNodeLabels then
    [ transform [ Rotate 90 labelX labelY ] ]
  else
    []
```

Aufgabe 9.3: Eigener Decoder für die Länderhierarchie

`countryHierarchy.json` hat ein anderes Format: Der Name steht verschachtelt unter `data.id`, und es gibt keine Werte an den Blättern. Dafür wird ein eigener Decoder geschrieben, der den Namen über `Decode.at` aus dem geschachtelten Feld liest:

```
countryDecoder : Decoder (Tree Node)
countryDecoder =
  Decode.map2
    (\name children ->
      Tree.tree (mkNode name Nothing) (Maybe.withDefault [] children))
    (Decode.at [ "data", "id" ] Decode.string)
    (Decode.maybe
      (Decode.field "children"
        (Decode.list (Decode.lazy (\_ -> countryDecoder)))))
```

Der resultierende Baum (World → Kontinent → Subregion → Land) wird mit demselben `treeView` dargestellt wie flare.

Lesbarkeit großer Bäume

flare (≈ 250 Knoten) und die Länderhierarchie (261 Knoten) sind deutlich größer als die Bäume aus 9.1. Zunächst wurden alle vier Bäume gleich behandelt und mit `Hierarchy.size 1700 850` gezeichnet. Bei den großen Bäumen funktioniert das nicht, weil die rund 220 Blätter von flare auf eine feste Breite von 1700 px gequetscht werden und Knoten wie Label dadurch übereinander liegen. Zusätzlich verkleinerte das SVG über `width: 100%` alles noch auf die Seitenbreite.

Der Grund liegt darin, dass `Hierarchy.size` und `Hierarchy.nodeSize` unterschiedlich arbeiten. `size` skaliert das fertige Layout in eine feste Box, was bei kleinen Bäumen passt, deren Größe man kennt. `nodeSize` dagegen gibt jedem Knoten einen festen Platz und lässt das Layout entsprechend wachsen. Für hunderte Knoten ist nur das Zweite sinnvoll. Die Knoten oder die Schrift einfach kleiner zu machen, hätte das Überlappen nur verschoben, denn die Ursache ist das Einpassen in eine feste Fläche.

Deshalb werden die beiden Fälle über das Feld `fixedSize : Maybe (Float, Float)` getrennt. Kleine Bäume behalten ihre feste Fläche, große Bäume lassen `Hierarchy.size` weg und wachsen über `nodeSize` und die Abstände natürlich.

```
layoutAttrs =
  (case config.fixedSize of
    Just ( w, h ) -> [ Hierarchy.size w h ]
    Nothing       -> [])
  ++ [ Hierarchy.nodeSize (\_ -> ( r * 2, r * 2 ))
    , Hierarchy.parentChildMargin config.parentChildMargin
    , Hierarchy.peerMargin config.peerMargin
    , Hierarchy.layered ]
```

Die `viewBox` wird anschließend aus den tatsächlichen Knoten-Grenzen (Minimum/Maximum von `x/y`) berechnet, mit zusätzlichem Platz unten für die gedrehten Labels. Das SVG wird in seiner echten Pixelgröße in einem scrollbaren Container gerendert, statt auf Seitenbreite herunterskaliert zu werden – so bleiben die Label lesbar und man scrollt durch den breiten Baum.

Baum	<code>Hierarchy.size</code>	Darstellung
9.1 binär / geordnet	1320×520 (fest)	skaliert in feste Fläche
9.2 flare / 9.3 Länder	– (natürlich)	echte Größe, scrollbar, Labels 90°